Agentic RAG Project Brief

Neural AI · RSS Panel · 8 members · 2 weeks · no model training

1. Problem statement

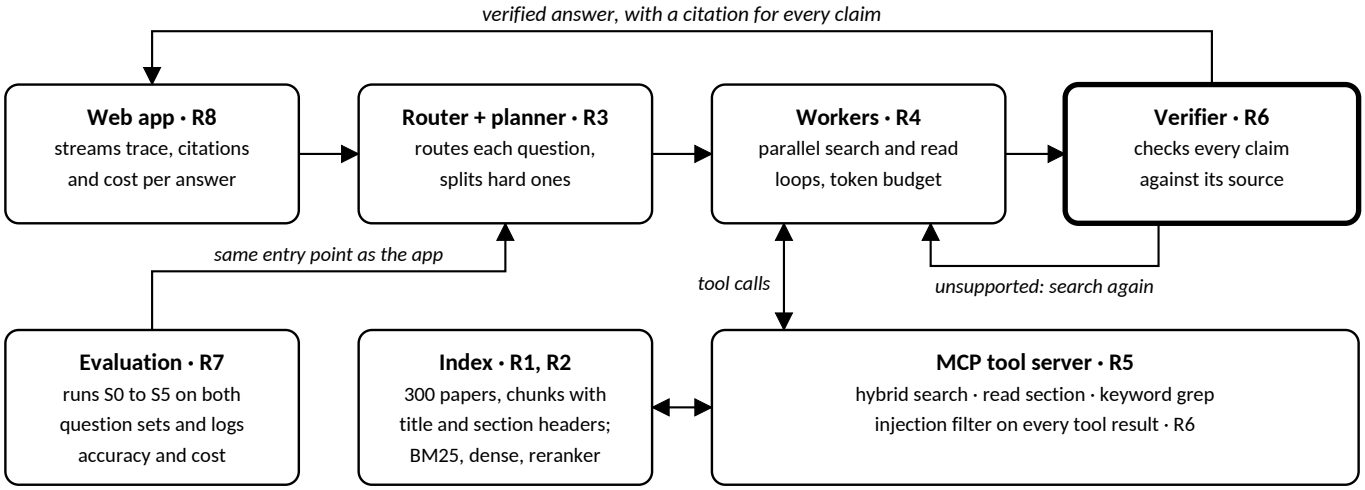
Build a research agent that answers questions about the latest AI papers, cites a supporting passage for every claim, and proves when agentic retrieval is worth its cost.

Standard RAG retrieves once and answers once. It breaks when the evidence is spread across several papers, when a question needs a comparison, or when the answer has to be proven. Deep research agents replace this with a loop: plan, search, read, verify, repeat. The loop now powers commercial research assistants, yet three questions about it are still open. This project tests all three on one system:

- **Cost:** when do extra retrieval steps pay for their latency and tokens, and can a router skip them when they do not?
- **Trust:** can every sentence of an answer be traced to a passage that actually supports it?
- **Safety:** can instructions planted inside a retrieved document hijack the agent, and what stops them?

Scope: about 300 recent arXiv papers on agents, retrieval and reasoning. No model training: free-tier LLM APIs or a small local model are enough, and the whole system runs on a laptop.

2. System architecture



Every module reads or writes one answer object (answer, citations, trace, cost), and nothing reaches the user without passing the verifier (bold). Suggested stack: Python, LangGraph, MCP Python SDK, Qwen3-Embedding-0.6B, bm25s, bge-reranker-v2-m3, LanceDB, FastAPI, Streamlit, Langfuse, Docker.

3. Research question and evaluation

Question: with the same model and budget, how much do planning, iteration and verification improve multi-hop answers over single-shot RAG, and what does each step cost?

Setup	What changes	Built by
S0 Closed book	No retrieval; the model answers from memory	R7
S1 Vanilla RAG	Dense top-5 chunks, one pass	R1, R2
S2 Tuned RAG	Hybrid BM25 plus dense search, reranker, context headers	R1, R2
S3 Single agent	Search, read and reflect loop, capped at 8 steps	R4
S4 Multi-agent	Planner, parallel workers and verifier; router on or off	R3, R4, R6
S5 Index-free agent	Keyword and grep tools only, no embeddings	R5

- **Data:** 80 custom questions over the corpus, 10 written by each member, with gold evidence and some unanswerable ones, plus 120 MuSiQue and HotpotQA questions from the A-RAG benchmark release.
- **Metrics:** exact match, F1 and LLM-judge accuracy; Recall@10; citation precision and recall; unsupported-claim rate; latency, tokens and cost per query; attack success on 20 planted documents.
- **Targets:** S4 beats S1 on multi-hop accuracy on both sets; the verifier at least halves unsupported claims; the router cuts cost per query by 30% or more while losing under 2 points of accuracy.
- **Rule:** one model at temperature 0 and one token budget per setup; every number ships with its cost.

4. Team: 8 equal roles in 4 pairs

All 8 work on one project with equal load: each owns one module end to end, and pairs review each other's code.

Pair	Role	Owns	Delivers
A · Knowledge	R1 Corpus	Fetch, parse and chunk papers; add headers	Versioned corpus, chunking ablation
	R2 Retrieval	BM25, dense, hybrid fusion, reranker	Search API, Recall@10 report
B · Agent	R3 Router and planner	Difficulty routing, sub-questions, stop rules	Router cost-accuracy curve
	R4 Workers and context	Tool loop, parallel workers, context budget	S3 and S4 agents
C · Tools and trust	R5 Tools and MCP	MCP server: search, read, keyword grep	S5 agent, external client test
	R6 Verification	Claim checks, abstention, injection filter	Faithfulness and attack results
D · Eval and product	R7 Evaluation	Eval sets, harness, judge rubric, ablations	Results table, error analysis
	R8 Product	FastAPI, Streamlit UI, tracing, run script	Working app, final deck

Everyone also: a 5-minute talk on their tagged paper; 10 eval questions written and 10 reviewed; tests and docs for their module; two slides in the final presentation. R7 leads the results story; R8 assembles the deck and runs the live app.

5. Deliverables

- Public GitHub repo: one command runs the app and reproduces the results table.
- Working app with live agent trace, clickable citations and cost per answer; MCP server any client can call.
- Results table for S0 to S5 and an error analysis of 20 failures.
- Final offline presentation, about 15 minutes, with the results and a live run of the app.

6. Feasibility in 2 weeks

- **Nothing is trained.** Every part is an off-the-shelf library (bm25s, LanceDB, LangGraph, MCP SDK) plus prompts.
- **Small corpus.** 300 papers index in about an hour on a laptop; headers are title and section, so no LLM calls.
- **Parallel by design.** Schemas are fixed first and modules are built against mocks, so no pair waits on another.
- **Bounded evaluation.** 200 questions across 6 setups is under 10,000 LLM calls, split across members' free-tier keys.
- **Even load.** Each role is about 20 hours of work, including its paper talk and eval questions.
- **Cut order if time runs short:** S5, then the router ablation; the core S4 versus S1 and verifier results still stand.

7. Core reading

Everyone reads the two surveys first; each role presents its tagged paper. Titles are clickable links.

Paper or post	Source	For	Why it matters here
A Survey on Agentic RAG, revised 2026	arXiv 2501.09136	All	Map of agent patterns: reflection, planning, tools
Survey of Deep Search Agents, ACL 2026	arXiv 2508.05668	All	How deep search agents are built and evaluated
Introducing Contextual Retrieval, 2024	Anthropic	R1	Context headers plus BM25 cut failed retrievals
Rethinking Reasoning-Intensive Retrieval, ACL 2026	arXiv 2605.04018	R2	Judge retrievers inside the agent loop
Adaptive-RAG, NAACL 2024	arXiv 2403.14403	R3	Route each question to the cheapest strategy
RAGRouter-Bench, 2026	arXiv 2602.00296	R3	Benchmark for RAG routing decisions
Search-R1, COLM 2025	arXiv 2503.09516	R4	Reasoning interleaved with search; skip the RL part
Effective context engineering for AI agents, 2025	Anthropic	R4	Keep context small as tool results pile up
A-RAG, 2026, with public code	arXiv 2602.03442	R5	Keyword, semantic and chunk-read tools for agents
Model Context Protocol spec and Python SDK	MCP docs	R5	The tool interface our server implements
Overcoming the Retrieval Barrier, 2026	arXiv 2601.07072	R6	Indirect prompt injection that survives retrieval
CRAG: Corrective RAG, 2024	arXiv 2401.15884	R6	Grade retrieved evidence before using it
BrowseComp-Plus, NeurIPS 2025	arXiv 2508.06600	R7	Fixed corpus separates retriever from agent quality
How we built our multi-agent research system, 2025	Anthropic	R8	Orchestrator and workers in production

New papers: github.com/YunjiaXi/Awesome-Search-Agent-Papers and github.com/DavidZWZ/Awesome-RAG-Reasoning.